Cahier des charges SAÉ : Projet Rust

1.0 Informations générales :

Groupe : Stras4Ever

Membres : Laurent Zhou, Pierre Mittelbronn, Maxime May, Mathieu Buannic

Nom du projet : Rusty Vault

1.1 Introduction et problématique :

Dans l'industrie logicielle, **la protection de la propriété intellectuelle** est extrêmement importante. Un binaire non protégé serait facile à analyser et donc les innovations obtenues grâce à la **R&D volées très facilement, des applications payantes ou encore des modifiées de manière non autorisée** distribuées librement**.**

2.0 Explication de l’état de l’art actuel :

Le domaine de la protection binaire est en **constante évolution depuis bientôt 50 ans**, c’est un domaine extrêmement riche en **technologie et créativité** de part sa nature du « **jeu du chat et de la souris** », qui nécessite une constante adaptation de la part des éditeurs, cependant, certains principes particulièrement importants restent communs parmi les principaux formats d’exécutables.

Il n’y a pas réellement de science exact sur comment protéger un binaire, mais bien **différente visions en fonction des éditeurs**, contraintes et obligations, car **chaque protection vient avec son lot de contraintes.**

2.1 Résumé rapides actuelles courantes et efficaces :

A. Obfuscation Statique (Renommage & Chiffrement de strings)

- **Capacités :** Supprime les symboles (noms de fonctions/variables) et chiffre les chaînes de caractères au repos. Empêche la lecture simple via la commande strings.
- **Limites :** Très fragile face à une analyse dynamique. Un débogueur permet de voir les chaînes déchiffrées en mémoire instantanément.

B. Packing (Compression/Chiffrement d'exécutable) - ex: UPX

- **Capacités :** Compresse le binaire et l'enveloppe dans un chargeur. Le code original n'est présent sur le disque que sous forme chiffrée.
- **Limites :** Une fois le binaire "dépaqueté" en mémoire au lancement, il redevient totalement vulnérable. Le "point d'entrée" (OEP) est facile à trouver pour un attaquant expérimenté.

C. Control Flow Flattening (OLLVM, Tigress)

- **Capacités :** Brise la logique linéaire du programme en utilisant un répartiteur central (dispatcher). Rend le graphe de contrôle (CFG) illisible.
- **Limites :** Peut être "nettoyé" par des outils de dé-obfuscation symbolique si l'algorithme de transition d'état est trop prévisible.

E. Virtualisation de code (VMProtect, Themida)

- **Capacités :** Transforme les instructions CPU natives en un bytecode propriétaire exécuté par une VM interne. C'est un des niveaux de protection le plus élevé.

- **Limites :** Impact significatif sur les performances (vitesse d'exécution divisée par 10 ou 100). Complexité de développement accrue.

Nous expliquerons ces technologies en détail dans la suite de ce document

3.0 L’obfuscation:

L’obfuscation est un terme général désignant une méthode ayant pour but de rendre du code moins compréhensible en conservant sa logique initiale

```
(A)
    function setText(data) {
    document.getElementById("myDiv").innerHTML = data;
    }
```

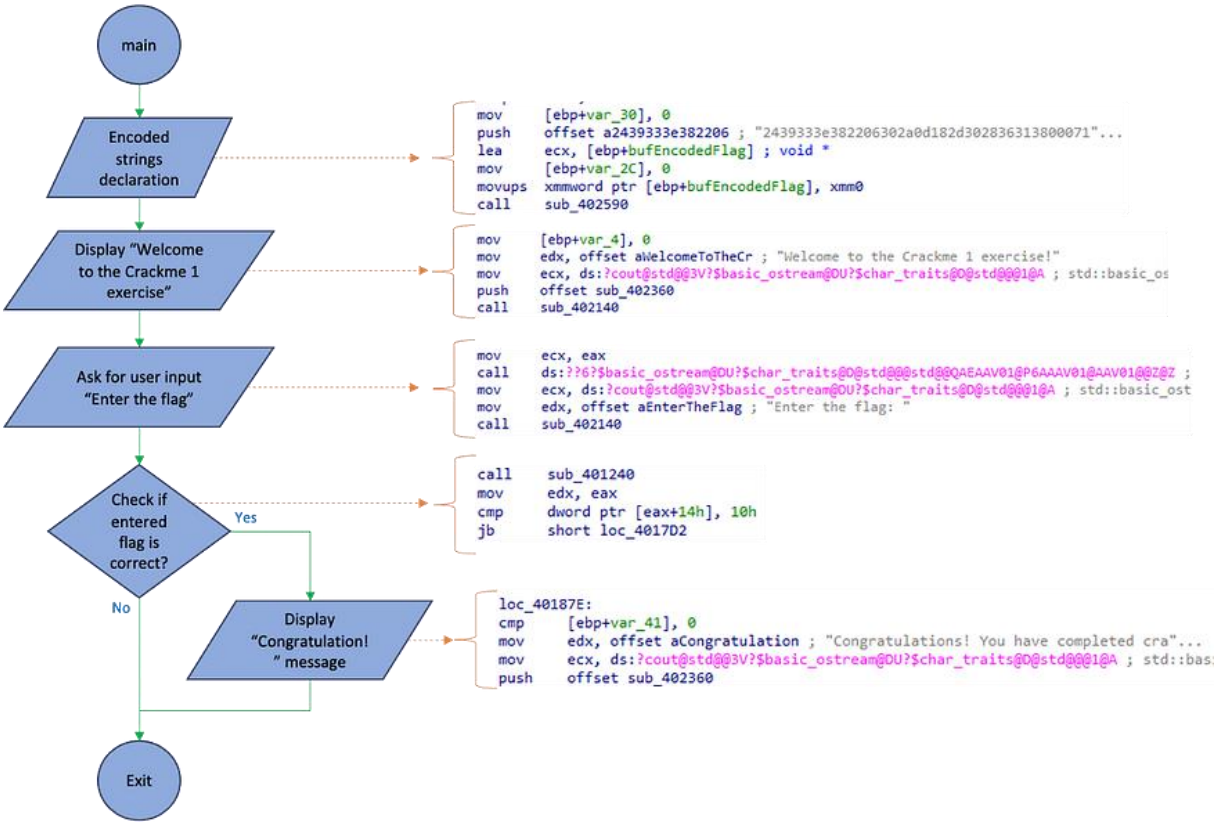
```
(B)
    function ghds3x(n) {
    h = "\x69\u006En\u0065r\x48T\u004DL";
    a="s c v o v d h e , n i";x=a.split(" ");b="gztxleWentBsyf";
 r=b.replace("z",x[7]).replace("x","E").replace("s","").replace("f","I")
    ["repl" + "ace"]("W","m")+"d";
c="my"+String.fromCharCode(68)+x[10]+"v";
    s=x[5]+x[3]+x[1]+"um"+x[7]+x[9]+"t";d=this[s][r](c);if(+![])
    { d[h]=n; } else { d[h]=c; } }
```

(A) Fonction en clair

(B) Fonction obfusquée

Pour cela on utilise des **noms de variables non intuitifs,** on rajoute des étapes et remplace les conditions par des **équivalents logiques plus complexes.** Rendre les strings non lisible par **altérations logique ou chiffrement** est aussi une bonne méthode, cela empêche un attaquant de comprendre un code lorsqu’il y a accès, mais aussi de se repérer dans un binaire compilé en maintenant l’**abstraction** apportée par la compilation. Cela rend donc intrinsèquement le code moins lisible et peut ralentir l’exécution.

Exemple avec le flot d’exécution d’un crackme très simple :

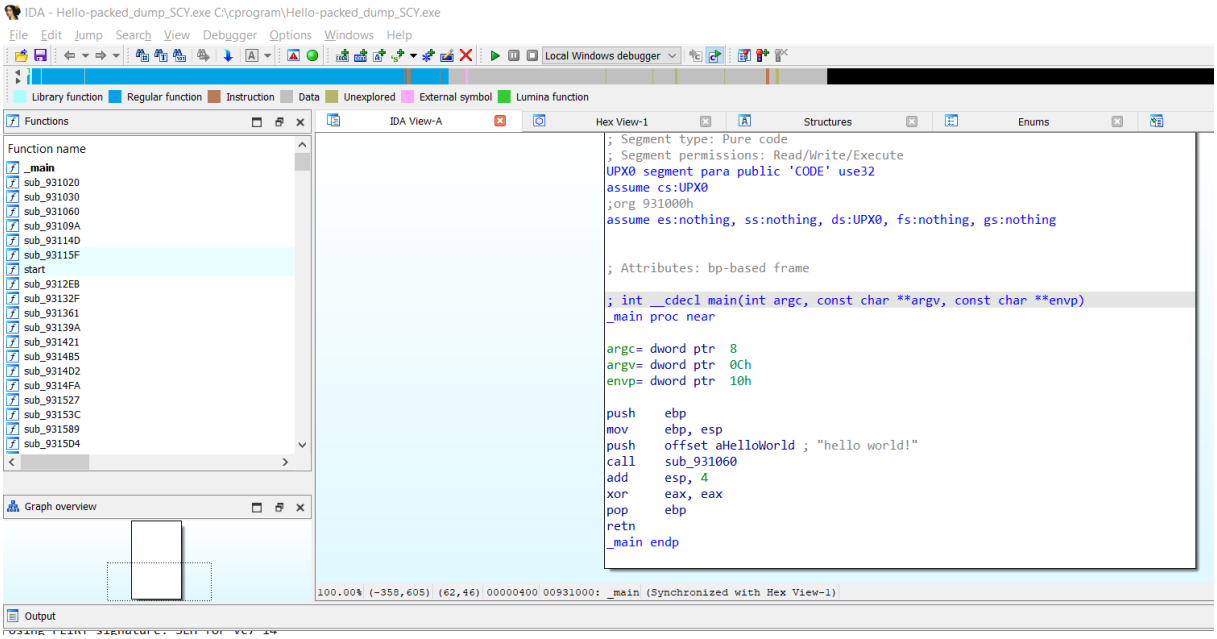


Ici si les strings aux offset aCongratulations ou aEntertheflag, l'attaquant aurait besoin d'un degré de compréhension supérieur pour savoir ou il se situe dans le binaire

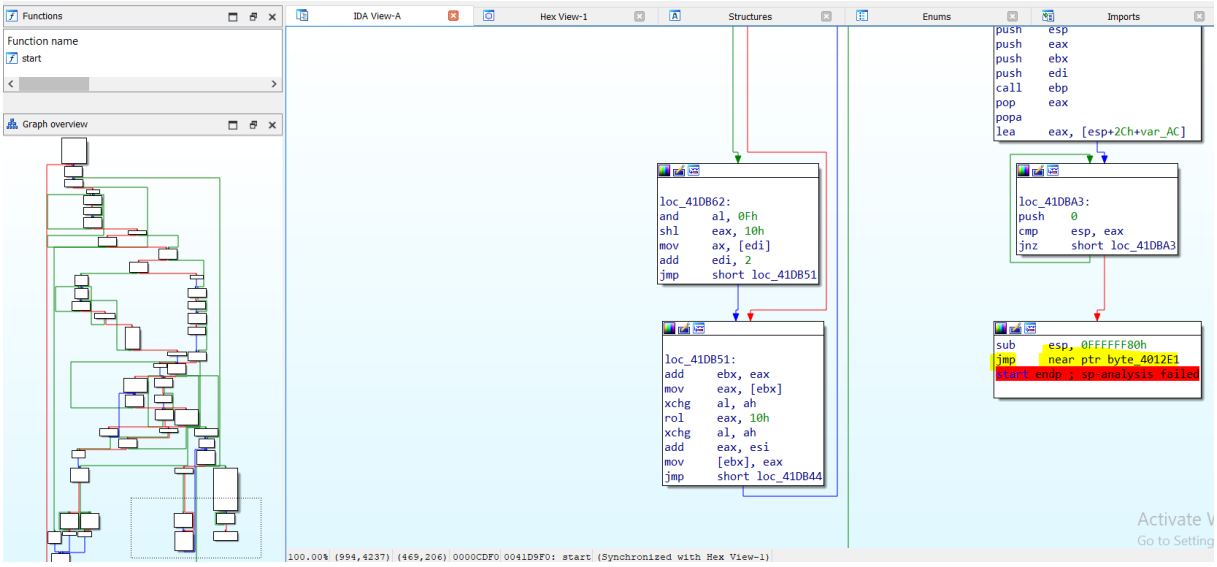
3.1 Packing, compression et chiffrement d'executable:

Le principe général est de **charger le binaire original sous forme de tableau dans un binaire** qui sert à placer le tableau dans un espace mémoire vide puis **d'appeler son point d'entrée original** pour déclencher l'exécution, le **code original n'est donc pas visible** si on analyse juste simplement le binaire cible.

On peut aussi appliquer des **transformations à ce tableau avant de le charger en mémoire** pour rendre l'analyse et la récupération plus difficile, par exemple XOR chaque byte avec une clé définie par l'utilisateur ou bien par une informations unique à chaque binaire (sa somme de contrôle MD5 par exemple)



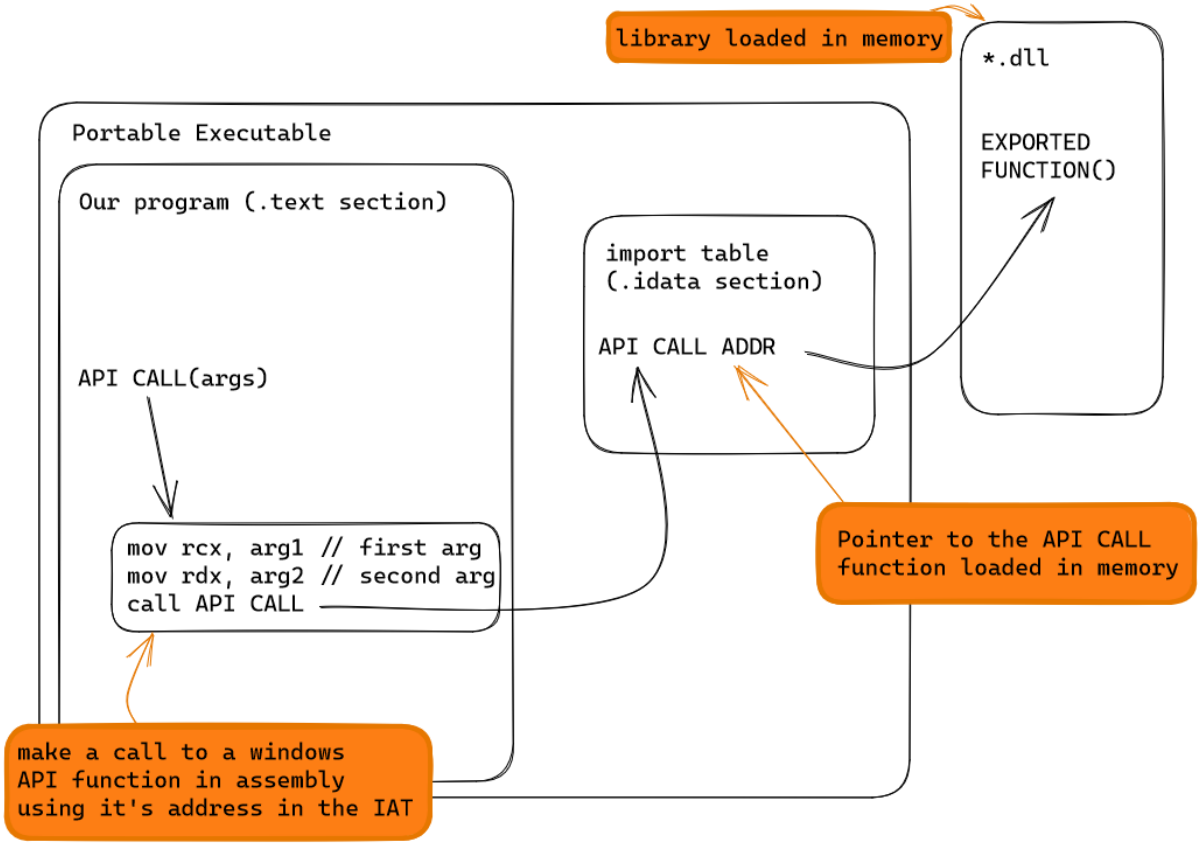
Ce simple programme hello world avec un graphe de contrôle unique, après un passage dans UPX (un packer très utilisé) devient:



Cela ne rend pas l’analyse impossible, ici si l’attaquant comprends que le dernier « jmp » est le point le binaire qui charge l’original saute vers son point d’entrée, il aurait juste à dump la taille de l’exécutable original à partir de l’adresse de base à laquelle le chargeur jump.

3.2 IAT/GOT Hooking:

La plupart des binaires utilisent des **librairies**, ces librairies sont la plupart chargées dynamiquement, c'est la que **l'IAT/GOT (Import adresse table et global offset table, selon le format du binaire)** rentre en jeu, **elle associe un symbole à son adresse mémoire.**



L'IAT/GOT sont des arrays null terminés présents dans notre binaire, ce qui veut dire que **rien n'empêche leur réécriture à l'exécution**

```
LibraryA = LoadLibraryA("shell32.dll");
IsUserAnAdmin = (BOOL (__stdcall *)())GetProcAddress(LibraryA, "IsUserAnAdmin");
CommandLineW = GetCommandLineW();
v15 = ((__int64 (__fastcall *)(LPWSTR, int *))IsUserAnAdmin)(CommandLineW, &v11);
if ( v11 <= 4 )
  return qword_7FF79FB28030(a1, a2, a3, a4);
MaxCount = wcslen(*(const wchar_t **)(v15 + 32));
v13 = MaxCount - 1;
v7 = alloca(sub_7FF79FB23850());
Dest = (char *)v9;
wcstombs((char *)v9, *(const wchar_t **)(v15 + 32), MaxCount);
Dest[MaxCount] = 0;
if ( !sub_7FF79FB21784(Dest) )
  return qword_7FF79FB28030(a1, a2, a3, a4);
```

Extrait d'une fonction qui hooké l'IAT en inversant IsUserAnAdmin et CommandLineToArgvW, en apparence **le code n'a plus de sens** étant donné que le cette fonction semble récupérer une chaine de caractère avant de la passer à une fonction qui applique des transformations dessus, **l'attaquant n'a donc pas de résultat probant**

```
strcpy(&Str2[14], "CommandLineToArgvW");
strcpy(Str2, "IsUserAnAdmin");
LibraryA = LoadLibraryA("shell32.dll");
v11 = LibraryA;
v10 = (char *)LibraryA + *((int *)LibraryA + 15);
v9 = (_DWORD *)((char *)LibraryA + *((unsigned int *)v10 + 34));
v8 = v9[6];
v7 = (char *)LibraryA + (unsigned int)v9[7];
v6 = (char *)LibraryA + (unsigned int)v9[8];
v5 = (char *)LibraryA + (unsigned int)v9[9];
for ( i = 0; i < v8; ++i )
{
  Str1 = (char *)LibraryA + *(unsigned int *)&v6[4 * i];
  if ( !strcmp(Str1, &Str2[14]) )
    v15 = *(_DWORD *)&v7[4 * *(unsigned __int16 *)&v5[2 * i]];
}
for ( j = 0; ; ++j )
{
  result = v8;
  if ( j >= v8 )
    break;
  v4 = (char *)LibraryA + *(unsigned int *)&v6[4 * j];
  if ( !strcmp(v4, Str2) )
  {
    VirtualProtect(&v7[4 * *(unsigned __int16 *)&v5[2 * j]], 8u, 4u, &flOldProtect);
    *(_DWORD *)&v7[4 * *(unsigned __int16 *)&v5[2 * j]] = v15;
    VirtualProtect(&v7[4 * *(unsigned __int16 *)&v5[2 * j]], 8u, flOldProtect, &flOldProtect);
  }
}
return result;
```

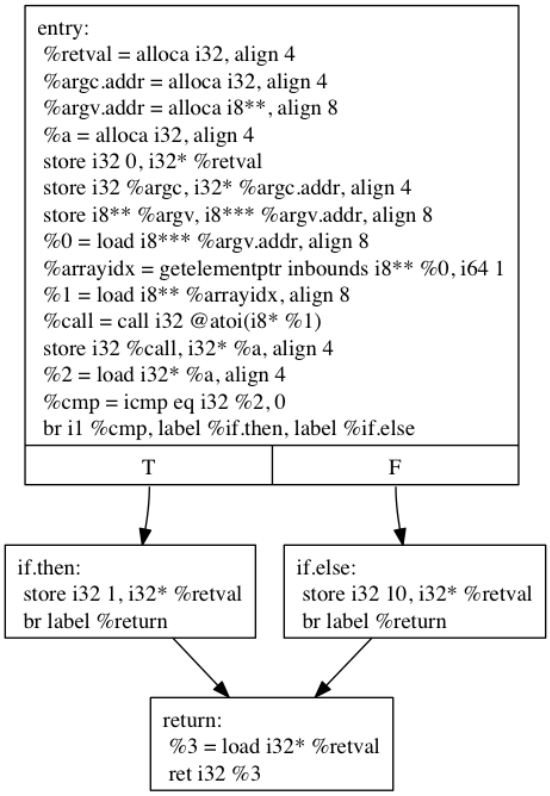
Code de la fonction d'inversion, on vient **charger la librairie, trouver l'adresse du symbole qu'on veut remplacer, puis on réécrit son adresse dans l'espace d'adressage virtuel.**

3.3 Control Flow Flattening:

Dans la même veine que l'obfuscation, il s'agit de rendre **le logique du code plus compliquée à comprendre en changeant son flux de contrôle**

Exemple ci-dessous avec le code suivant et son graphe :

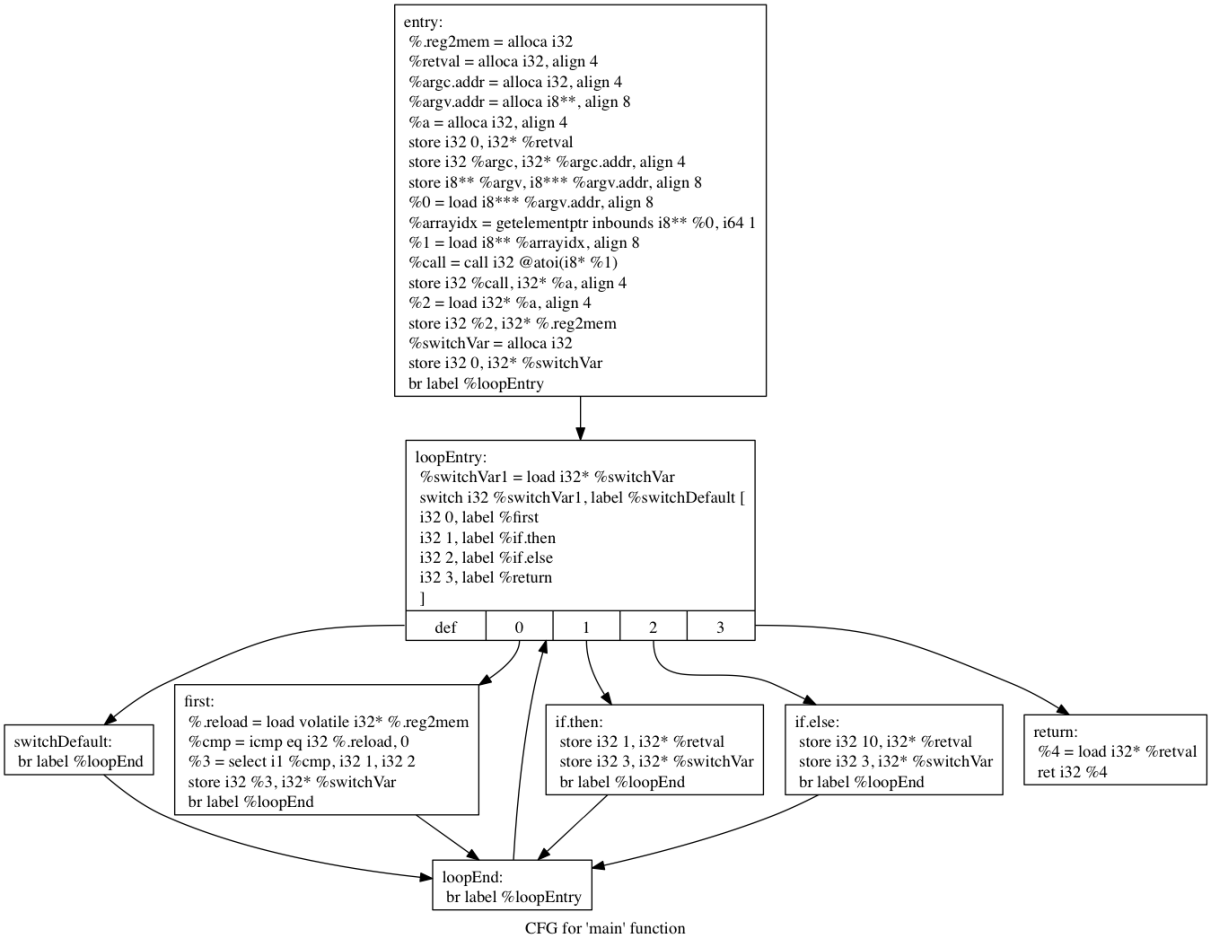
```c
#include <stdlib.h>
int main(int argc, char** argv) {
  int a = atoi(argv[1]);
  if(a == 0)
    return 1;
  else
    return 10;
  return 0;
}
```



CFG for 'main' function

Après un passage dans ollvm (version de LLVM spécialisée dans l’obfuscation), le code devient

```c
#include <stdlib.h>
int main(int argc, char** argv) {
  int a = atoi(argv[1]);
  int b = 0;
  while(1) {
    switch(b) {
      case 0:
        if(a == 0)
          b = 1;
        else
          b = 2;
        break;
      case 1:
        return 1;
      case 2:
        return 10;
      default:
        break;
    }
  }
  return 0;
}
```



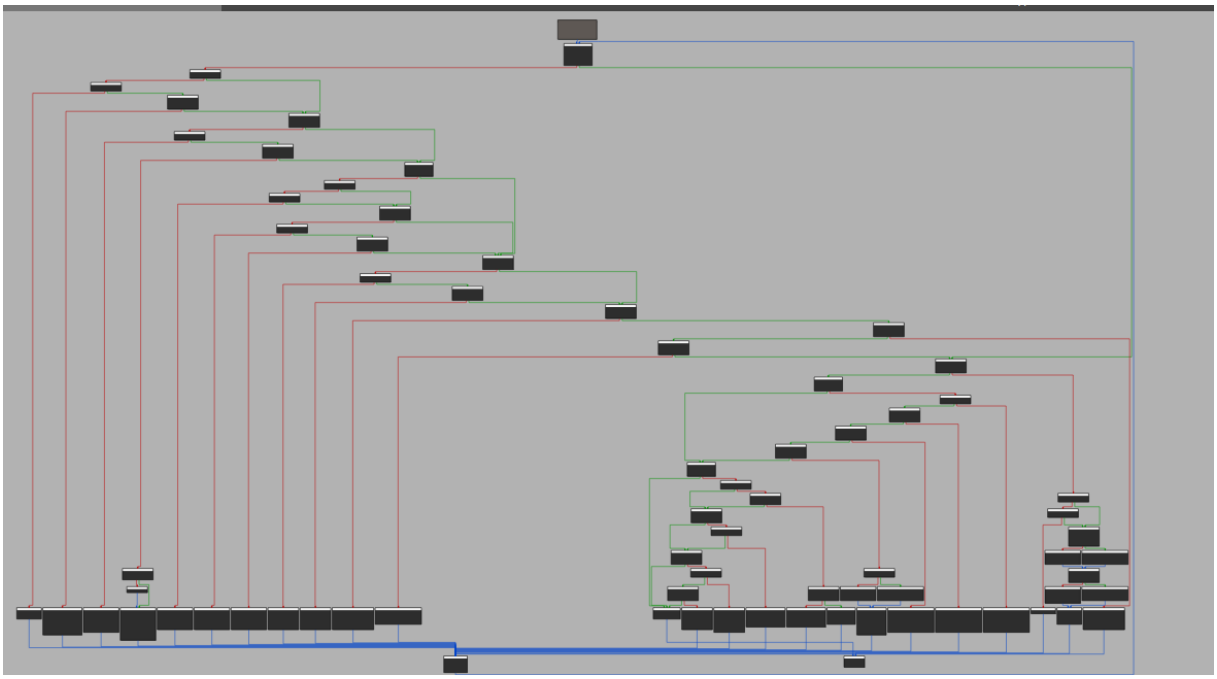
CFG for 'main' function

L’ordre des opérations est beaucoup plus abstrait et difficile à suivre.

3.4 Virtualisation:

Un des **niveau de protection les plus avancés,** bien que ce principe ne soit pas si récent que ça. On revient au **même principe que le packing,** c’est-à-dire charger notre exécutable en tant que tableau par un autre binaire. Mais cette fois le fonctionnement différé un peu, au lieu d’avoir le binaire original, le tableau stocké ne contient **pas le bytecode original mais son équivalent dans un bytecode propriétaire** ou bien différent de l’architecture d’origine

Exemple extrait d’un crackme utilisant une VM z80 :



```
for ( i = byte_8049A95; ; i = v1 - 3 )
```

Itération sur le tableau contenant le code original

```
.data:08049A95 byte_8049A95    db 0C3h, 0, 1, 0FFh dup(0), 26h, 20h, 3Eh, 0, 1, 42h, 1
.data:08049A95                                             ; DATA XREF: sub_8048436↑o
.data:08049A95                                             ; sub_8048436+57↑o ...
.data:08049B9E                 db 87h, 3Ch, 2, 3, 0FEh, 0FFh, 0C2h, 13h, 1, 3Ch, 25h
.data:08049BA9                 db 0C2h, 9, 1, 2 dup(0), 6, 20h, 0Eh, 0, 1Eh, 2Ah, 21h
.data:08049BB5                 db 3, 0, 9, 2Bh, 7Eh, 57h, 21h, 42h, 1, 9, 2Bh, 7Eh, 0AAh
.data:08049BC2                 db 93h, 1Dh, 21h, 62h, 1, 9, 2Bh, 0BEh, 0C2h, 3Ch, 1, 5
.data:08049BCE                 db 0C2h, 1Fh, 1, 2 dup(0), 76h, 0FFh, 0EEh, 0CCh, 20h dup(0)
.data:08049BF7                 db 3Dh, 50h, 3Eh, 31h, 5Ah, 0FCh, 0Fh, 93h, 26h, 1Fh, 43h
.data:08049C02                 db 2Ah, 58h, 44h, 0F7h, 81h, 17h, 54h, 3Bh, 10h, 0, 1Fh
.data:08049C0D                 db 5Eh, 0DDh, 0DEh, 0DFh, 0E2h, 0E9h, 0F8h, 17h, 56h, 0D5h
.data:08049C17                 db 100h dup(0)
```

Programme original sous forme d’opcode Z80

```
v1 = i + 4;
result = sub_80488F0();
v4 = 0;
if ( byte_8049A90 )
  goto LABEL_35;
if ( byte_8049A92 )
  break;
if ( byte_8049A91 )
{
  if ( byte_8049A91 != 3 )
    break;
  word_8049A8E += SBYTE1(result);
  word_8049A8E += 2;
  i = &byte_8049A95[(unsigned __int16)word_8049A8E];
}
else
{
  ++word_8049A8E;
  i = v1 - 3;
}
```

Utilisation des registres (on peut voir que V1 est le program counter car incrémenté de 4 bits à chaque boucle comme le ferais un processeur) et le reste, un énorme bloc de if avec chaque opcode et quoi faire en fonction de l’état des registres

Cet exemple est extrêmement simplifié, les VM utilisées en production comme VMProtect ou Themida ont des **opcodes pseudos aléatoire** qu'ils **dérivent en fonction de facteurs non connus et dépendants de leur version.**

Bien que très fastidieux à analyser, cela reste possible et cette technologie n'est pas sans conséquences, la vitesse d'exécution est fortement ralentie.

3.5 Anti-Debug:

L'anti-debug a pour principe **d'empêcher l'exécution d'un binaire protégé dans un environnement contrôlé** par l'attaquant pour rendre plus difficile son analyse au moment de l'exécution. On y retrouve des fonctions d'api windows, comme IsDebuggerPresent(), CheckRemoteDebuggerPresent() , NtQueryInformationProcess()… et de nombreuses autres

Mais aussi d'autres techniques un peu plus discrètes avec des fonctions changeant de comportement lorsqu'attachées à un debugger, **la vérification de la marque du processeur ou de l'os pour vérifier que l'environnement n'est pas une VM, la vérification des timing d'exécution** (l'exécution dans un debugger est toujours plus lente), ou encore avec des instructions pièges pour le debugger (int 3).

Exemple :

```
LONG UnhandledExceptionFilter(PEXCEPTION_POINTERS pExceptionInfo)
{
    PCONTEXT ctx = pExceptionInfo->ContextRecord;
    ctx->Eip += 3; // Skip \xCC\xEB\x??
    return EXCEPTION_CONTINUE_EXECUTION;
}

bool Check()
{
    bool bDebugged = true;
    SetUnhandledExceptionFilter((LPTOP_LEVEL_EXCEPTION_FILTER)UnhandledExceptionFilter);
    __asm
    {
        int 3                       // CC
        jmp near being_debugged     // EB ??
    }
    bDebugged = false;

being_debugged:
    return bDebugged;
}
```

Si jamais un debugger est attaché int3 sera pris -> Donc détecté

On peut aussi stocker la somme de contrôle MD5 de toute les instructions et la vérifier en permanence, si le debugger met un point d'arrêt (remplace l'instruction au pc par un int3), alors on crash le programme.

Il existe de multiple méthodes, cette notion n'a pas réellement de défaut si ce n'est qu'elle rend le développement plus complexe.

4.0 Architecture et notions retenues:

Ce projet pourra présenter plusieurs niveau de complexité technique dépendamment de notre rapidité à l'implémenter.

Technologies utilisées par notre projet par ordre de priorité:

- **Packing** : Packer basique pour un code source Rust ou binaire compilé
- **Obfuscation** : Parsing du code, Suppression des symboles, Renaming des variables, chiffrement des strings (algorithme et critère de dérivation de la clé encore à déterminer)
- **Anti-Debug** : Recherches sur les api disponibles, et des indicateurs de debugging (flag oracle virtual box, vmware, IlSpy, x64dbg, etc…)

Implémentation optionnelles en fonction de la vitesse d'avancement :

- **Control Flow Flattening :** Implémentation custom pour faire fonctionner ollvm avec rustc et de ses différents profils
- **IAT/GOT hooking** : Parsing de l'IAT/GOT, modification des offsets au runtime
- **Virtualisation** : Implémentation d'une machine virtuelle pour packer le binaire

4.1 Planning:

Le planning est calqué sur les deux échéances majeures de la SAÉ. (MVP = minimum viable product)

Phase 1 : Consolidation du MVP (Soutenance de Mars)

Objectif : Présenter un binaire protégé de manière statique et capable de se charger seul en mémoire.

- **Février 2026 :** Étude approfondie des formats de fichiers et début du développement du **Packer.**
    - Mise en place du module de **chiffrement des strings**
- **Mars 2026 (Début) :** Intégration des premières fonctions **Anti-Debug** pour bloquer les outils d'analyse simple
    - Tests de stabilité du chargement mémoire.
- **23 - 27 MARS 2026 : PREMIÈRE SOUTENANCE**

Phase 2 : Extension et Optimisation (Soutenance Finale de Mai)

Objectif : Ajouter les couches de protection complexes en fonction du temps restant.

- **Avril 2026 :** Développement du **Control Flow Flattening**
    - Priorité 2 : Étude de faisabilité de la **Virtualisation**
- **Mai 2026 (Début) :** Implémentation optionnelle du **IAT/GOT Hooking** pour brouiller les pistes système
    - Tests intensifs de résistance et mesures de l'impact sur les performances
- **18 - 22 MAI 2026 : SOUTENANCE FINALE**

4.3 Répartitions des taches:

A déterminer.

5.0 Ressources et inspiration:

Control Flow Flattening · obfuscator-llvm/obfuscator Wiki

Home - Unprotect Project

Malware - IAT Hooking | TRIKKSS Blog

Exploring the Export Table [Windows PE Internals] - DEV Community

Challenges/Cracking : ELF x64 - Rust backdoor [Root Me : plateforme d'apprentissage dédiée au Hacking et à la Sécurité de l'Information]

Taming a wild nanomite-protected MIPS binary with symbolic execution: No Such Crackme

Executable compression - Wikipedia

Encrypting Strings to Evade Static Detection | by S12 - 0x12Dark Development | Medium

Anti-Debug Tricks
Quick look around VMP 3.x - Part 1 : Unpacking | r0da's Blog
Unpack Themida

wwh1004/ollvm-16: Obfuscator-LLVM for LLVM 16.x branch